La IA.
El entrenamiento.
El deployment.

Cómo se aprende un evaluador de ajedrez, cómo se conserva el campeón y cómo se publica como una API.

14,828,801
parámetros por red neuronal

2 notebooks
entrenamiento y despliegue independientes



Alcance

Este documento describe la implementación local de **Chess**: el modo neuronal predeterminado, su persistencia y el servicio de inferencia. La interfaz del tablero queda fuera del alcance, salvo su conexión con la API. Los nombres internos `cristopher_*` se conservan por compatibilidad con archivos y modelos.

Contenido

02 · Representación y arquitectura
03 · Aprendizaje por evolución
04 · CPU, GPU e inferencia
05 · Checkpoints y campeón
06 · Pipeline de deployment
07 · Contrato HTTP
08 · Operación y recuperación
09 · Verificación y mapa del código

> **Lectura del resultado:** más parámetros aumentan capacidad y coste. El tamaño de la red, las pruebas de legalidad y el tiempo de entrenamiento no demuestran por sí solos una fuerza de juego ni un Elo.

De una posición a una puntuación

La red es un **perceptrón multicapa (MLP)** con dos capas ocultas ReLU y una salida escalar. Evalúa posiciones; las jugadas legales las genera el motor de reglas. Para elegir una jugada se evalúan las posiciones resultantes y se comparan sus valores.

```
784 → 4608 [ReLU] → 2432 [ReLU] → 1
```

Capa	Pesos + sesgos	Parámetros
Entrada → oculta 1	784 × 4608 + 4608	3,617,280
Oculta 1 → oculta 2	4608 × 2432 + 2432	11,209,088
Oculta 2 → salida	2432 × 1 + 1	2,433
Total	Σ (entradas + 1) × salidas	**14,828,801**

Los 784 valores de entrada

Índices	Significado
0–767	12 planos × 64 casillas: seis tipos de piezas propias y seis rivales. Si juegan negras, se refleja el tablero verticalmente para orientar la posición al jugador que mueve.
768–771	Cuatro derechos de enroque: corto/largo propio y rival.
772–779	Columna de captura al paso, solo cuando existe una captura legal.
780	Contador de medias jugadas sin captura ni movimiento de peón: min(100, contador) / 100.
781–783	Ceros de relleno para alinear las dimensiones.

La red corrige una evaluación base

```
V(posición) = x · prior + 10 + 600 · tanh(MLP(x))
```

El resultado interno se expresa en **centipeones desde el lado que mueve**. El *prior* aporta material y tablas pieza-casilla (PST); el término neuronal queda limitado a ±600 centipeones. El +10 es el término fijo de iniciativa. Las posiciones terminales se puntúan con sus reglas de mate o tablas.

Si existe el campeón clásico de `full_v1`, se toma su material y el promedio de sus PST. En caso contrario se usan valores predefinidos. La capa final inicia en cero: la primera red produce la evaluación base antes de que la evolución modifique sus pesos.

> **Tamaño de los pesos:** 59,315,204 bytes en FP32 (59.3 MB) o 29,657,602 bytes en FP16 (29.7 MB), sin metadatos. Es aproximadamente 4× la red anterior de 3,706,881 parámetros. El consumo total de entrenamiento es mayor.

Evolución a partir de partidas

El modo predeterminado usa **estrategias de evolución con perturbaciones simétricas**. No emplea retropropagación ni etiquetas de Stockfish: evalúa variantes de los pesos mediante resultados de partidas y desplaza el centro de la población hacia las variantes mejor clasificadas.

1. **Crear candidatos.** Se generan 32 vectores de ruido gaussiano ε. Cada uno produce dos candidatos, μ + σ·s·ε y μ − σ·s·ε: 64 redes, además del centro μ. La escala s depende del tamaño de entrada de cada capa.
2. **Jugar enfrentamientos.** Cada candidato y el centro juegan aperturas compartidas, con colores invertidos. Los rivales incluyen bots aleatorios, la red inicial, un compañero de población, el campeón congelado y, cuando existen, antiguos campeones.
3. **Calcular fitness.** Victoria = 1, tablas o partida truncada = 0.5, derrota = 0. Se añade a las victorias un bono de hasta 0.01 por terminar antes. El fitness del candidato es la media de sus partidas.
4. **Actualizar el centro.** El fitness se transforma en una utilidad por rango entre −0.5 y +0.5. Los empates reciben la misma utilidad. Se combinan las diferencias entre cada par positivo/negativo para obtener la dirección de actualización.
5. **Evaluar la promoción.** El candidato con mayor fitness, incluido el centro, sustituye al campeón únicamente si todas sus partidas contra este terminaron y su puntuación media en ellas supera 0.5.

$$g = \Sigma\ (u_i^{+} - u_i^{-}) \cdot \varepsilon_i \ /\ (32 \cdot \sigma)$$

$$\mu \text{ siguiente} = \mu + 0.03 \cdot s \cdot \text{clip}(g, -1, 1)$$

σ comienza en 0.1. Se multiplica por 1.05 si la desviación del fitness es menor que 0.02, o por 0.995 en otro caso; queda limitada al intervalo [0.02, 0.5]. El centro puede cambiar aunque no se haya promovido un campeón.

Configuración de las partidas

- Dos pares por rival: cuatro partidas por emparejamiento, alternando colores.
- Aperturas aleatorias compartidas de 4–6 medias jugadas. El libro de aperturas está vacío.
- Selección neuronal de una sola media jugada de profundidad; exploración por temperatura 30 durante las primeras 24 medias jugadas.
- Límite de 256 medias jugadas. Las partidas truncadas no permiten superar la condición de promoción.
- Stockfish aparece cada cinco generaciones, incluida la generación cero, con fuerza limitada y un presupuesto de 0.03 s por jugada. Actúa como rival.

> **Campeón seleccionado:** es el que supera la condición implementada; no equivale al modelo objetivamente más fuerte. `champion_generation = -1` indica que aún se conserva el modelo inicial. La fuerza requiere una evaluación independiente.

Qué hacen CPU y GPU

CPU: reglas y preparación

Reconstruye posiciones desde el historial, genera sucesores legales, detecta terminales y codifica características. Los procesos usan `spawn` y un hilo BLAS por trabajador. La cantidad de trabajadores se adapta a núcleos, cuotas y una estimación de RAM.

GPU: evaluación y evolución

Agrupa posiciones por red y ejecuta multiplicaciones matriciales por lotes con `torch.bmm`. Usa FP16 para los forward en CUDA y FP32 para estado y actualizaciones ES. Mientras evalúa un grupo, la CPU puede preparar otro.

Un auxiliar C++ basado en una revisión fijada de *Disservin chess-library* acelera la preparación. Si no se compila, permanece la ruta Python. Los procesos de preparación no inicializan CUDA.

Memoria: una red no es toda la población

Componente	Magnitud orientativa con la red actual
64 candidatos, solo pesos FP16	≈1.90 GB decimales
32 vectores de ruido FP32	≈1.90 GB decimales
Centro, campeón, inicial y hasta cuatro históricos	Redes adicionales, más estado FP32
Activaciones y construcción del banco	Memoria adicional; existen copias y tensores temporales

Los parámetros de memoria usan un objetivo suave de 10×2^{30} bytes, una reserva de 3×2^{30} y lotes de hasta 4096 filas. La evaluación reduce el lote ante exceso de presupuesto u OOM. **Reducir el lote no reduce los pesos de la población**; un fallo al construirla puede requerir otra GPU o una población menor en un experimento nuevo.

Inferencia del campeón exportado

Dificultad	Selección	Presupuesto interno
easy	Una media jugada; muestreo entre las cinco mejores o, con probabilidad 0.12, entre todas.	0.15 s
medium	Búsqueda por lotes hasta 2 medias jugadas; ancho 4 debajo de la raíz.	0.5 s
hard	Búsqueda por lotes hasta 3 medias jugadas; ancho 8 debajo de la raíz.	2.5 s

En la raíz se consideran todas las jugadas legales. Bajo ella se restringe el ancho usando la evaluación base. El mate inmediato tiene prioridad; se penaliza repetir cuando se va por delante y existe alternativa. Los plazos se comprueban entre expansiones: un lote en curso puede exceder el tiempo interno.

> **Entrenamiento:** exige CUDA por defecto. **Deployment:** admite CPU o CUDA según disponibilidad. Los tiempos y la memoria de la red actual deben medirse en el runtime utilizado; aquí no se presenta un benchmark de Colab.

Reanudar y publicar son dos tareas

Todos los archivos persistentes se guardan bajo `/content/drive/MyDrive/cristopher_evo/`. El entrenamiento conserva un estado reanudable y publica una copia independiente del campeón para inferencia.

Checkpoint de entrenamiento	Bundle de deployment
Centro ES, campeón, red inicial, históricos, σ, semillas de perturbación, estado aleatorio, prior, configuración, generación y partidas en curso.	Código del motor, metadatos/configuración, pesos del campeón y hashes de integridad.
Permite continuar la evolución.	Permite cargar el modelo sin entrenar.
Se conservan los tres últimos completos.	Las carpetas publicadas anteriores permanecen disponibles.

```
checkpoints/gpu_es_15m_v1/gpu_XXXXXXXX.pt
logs/gpu_es_15m_v1/gpu_gXXXXXX/game_XXXXX.json
pgn/gpu_es_15m_v1/gpu_gXXXXXX/game_XXXXX.pgn
models/latest_champion.json
models/deployments/<id>/
  cristopher_engine.py
  cristopher_genome.json
  weights.pt
  manifest.json
```

Cuándo se guarda

Se crea un checkpoint y se publica el campeón inicial antes de jugar. Durante el entrenamiento se guarda aproximadamente cada 60 segundos, al terminar el lote actual; también en límites de generación y en una salida recuperable. El bloque `finally` intenta guardar al acabar o ante un error. Cada partida finalizada genera su registro JSON y PGN.

Publicación del campeón

1. Se escribe el checkpoint en un temporal, se vacía el búfer, se ejecuta `fsync` y se reemplaza el destino.
2. Se calcula una identidad del campeón, configuración relevante y código. Si coincide con una publicación íntegra, se reutiliza.
3. Se escribe una nueva carpeta con motor, pesos, metadatos y manifiesto SHA-256.
4. Solo al final se reemplaza `latest_champion.json`. El deployment sigue ese puntero y verifica los archivos antes de importar el motor.

Si el último checkpoint es ilegible, se prueba uno anterior. Una firma incompatible detiene la recuperación; si ninguno es legible, se informa del error sin comenzar silenciosamente desde cero.

> **Ante una desconexión abrupta:** el proceso no puede ejecutar un guardado final. Se recupera lo que haya terminado de escribirse y persistir en Drive; se puede perder trabajo posterior. La sustitución atómica organiza la publicación, pero no promete sincronización instantánea del servicio de Drive.

Usar un runtime por nombre de ejecución. El valor actual es `gpu_es_15m_v1`; las redes anteriores conservan `gpu_es_3m_v1` y `gpu_es_v1`. Reanudar requiere su arquitectura y configuración compatibles.

Del archivo guardado a la URL

`vendor/deploy.ipynb` funciona de forma independiente. **Run all** ejecuta cuatro etapas y mantiene el servicio en segundo plano después de que termina la última celda.

1. Cargar el campeón

Instala dependencias, monta Drive y lee `models/latest_champion.json`. Comprueba que la carpeta resuelta esté dentro de `models` y verifica sus hashes. Si no existe el puntero, admite el formato antiguo con archivos directamente en `models`. Instala PyTorch si se necesita y falta, importa el motor e instancia un único `API_ENGINE`.

2. Construir la aplicación

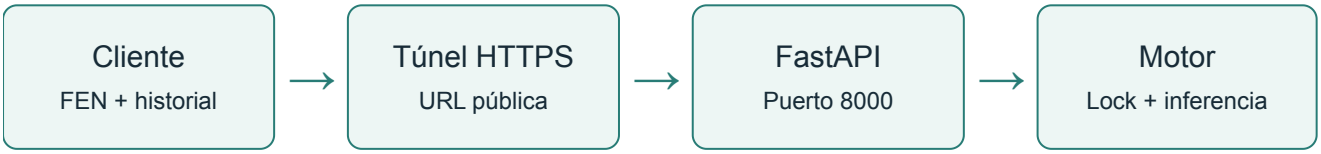
FastAPI define `/health`, `/move` y `/stats`. Pydantic valida solicitudes; `python-chess` verifica posición e historial. Un bloqueo protege el motor compartido y el trabajo se envía a un pool de hilos. CORS admite todos los orígenes, métodos y cabeceras.

3. Iniciar y probar el servidor local

Uvicorn escucha en `0.0.0.0:8000`, desde un hilo daemon. El notebook espera a que esté listo y prueba salud, una jugada legal desde la posición inicial, FEN inválido y posición finalizada. Los dos últimos deben responder HTTP 400.

4. Abrir y probar el túnel

La opción predeterminada ejecuta `cloudflared tunnel --url http://localhost:8000` y recoge la URL HTTPS de su salida. La alternativa ngrok usa `NGROK_TOKEN` de los secretos de Colab o un ingreso oculto. Por la URL pública se repiten las pruebas de salud y jugada legal, con hasta seis intentos. Solo después de aprobarlas se muestra la URL final.



Ciclo de vida

Reejecutar el deployment detiene su servidor y túnel previos y vuelve a cargar el campeón publicado. No recarga pesos automáticamente al cambiar el puntero en Drive. `shutdown_api()` detiene ambos componentes. Si el runtime termina, también termina el servicio; la URL puede cambiar al abrir otro túnel.

> **Alcance del servicio actual:** no incorpora autenticación ni limitación de solicitudes. La URL permite acceder a la API y `/stats` expone los últimos registros de jugadas. Está diseñado como despliegue temporal de notebook, sin garantía de disponibilidad continua.

Contrato y ejecución de una jugada

Ruta	Respuesta / función
GET /health	Estado, identificador del motor, versión del artefacto y segundos de actividad. Para el modo GPU actual: `cristopher-gpu-es-1`.
POST /move	Recibe FEN, historial UCI y dificultad; devuelve una jugada UCI con evaluación, profundidad, nodos y tiempo.
GET /stats	Contadores y las últimas 50 solicitudes de jugada, incluidas las fallidas. Los datos se mantienen en memoria.

Ejemplo de solicitud

```
POST /move
Content-Type: application/json

{
  "fen": "rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq - 0 1",
  "history": ["e2e4"],
  "difficulty": "medium"
}
```

FEN debe tener seis campos y representar una posición válida. Si el historial no está vacío, se reproduce desde la posición inicial estándar y debe llegar al mismo FEN. Para una posición aislada, se puede omitir el historial; en ese caso no se dispone de toda la información de repetición previa.

```
{
  "move": "e7e5",
  "info": {"eval": 0.35, "depth": 2,
           "nodes": 120, "time_ms": 80}
}
```

Respuesta ilustrativa: los valores no son una medición. En HTTP, `eval` se convierte a peones desde la perspectiva de blancas: +0.35 favorece a blancas. Una promoción lleva sufijo UCI, por ejemplo `a7a8q`.

Plazos, concurrencia y validación final

Los límites HTTP de easy/medium/hard son **3 / 10 / 25 segundos**, incluida la espera por el lock. Si se alcanza el límite se devuelve 504. Una operación nativa de NumPy o PyTorch puede seguir hasta terminar; conserva el bloqueo para impedir que dos llamadas usen simultáneamente el motor. Cancelar desde el cliente no mata esa operación.

La API comprueba la legalidad de la respuesta. Si el motor entrega una jugada ilegal, intenta una búsqueda de una media jugada sobre todos los sucesores legales y añade `fallback: true`. Ese respaldo usa la evaluación del motor. El cliente también valida la jugada antes de aplicarla.

Código	Causa
400	FEN inválido, partida finalizada o historial incompatible/ilegal.
422	Campos o dificultad inválidos; formato de historial no aceptado.
504 / 500	Plazo agotado / error de ejecución del motor.

Los errores usan `{"error": "descripción"}`. La versión identifica el formato del motor; no es un identificador único de cada campeón.

Ejecutar y recuperar el sistema

Entrenamiento: Run all

1. Abrir `vendor/train.ipynb` en Colab y seleccionar un runtime con GPU.
2. Dejar `TRAINING_MODE = gpu_neuroevolution`, `TRAINING_HOURS = 3` y `RUN_NAME` vacío para usar `gpu_es_15m_v1`.
3. Ejecutar todo y autorizar Drive. Se preparan dependencias, motor y trabajadores; comienza o se reanuda la evolución.
4. Al terminar, se exporta/verifica el campeón y se prueba su legalidad en un proceso nuevo. El presupuesto de tres horas corresponde al entrenamiento; la instalación y verificación añaden tiempo.

El máximo configurado es de 10,000 generaciones, pero la sesión suele terminar por presupuesto de tiempo. Run all posterior continúa el mismo experimento. Los formularios y metadatos ocultan inicialmente el código; se puede expandir cada celda para inspeccionarlo.

Deployment: Run all

1. Abrir `vendor/deploy.ipynb` en otro runtime con acceso al mismo Drive. Puede usar CPU.
2. Elegir `cloudflare` o `ngrok`, ejecutar todo y esperar las pruebas públicas.
3. Copiar la URL HTTPS final. En el cliente: Settings → Connect to Cristopher → Server URL; activar Use AI server, seleccionar AI y probar la conexión.
4. Mantener activo el runtime de deployment. Para cargar un campeón recién publicado, repetir Run all. Para detener el servicio, ejecutar `shutdown_api()`.

Los chequeos automáticos del cliente se realizan solo cuando el servidor está habilitado, el rival es IA y la pestaña está visible. El botón de prueba permite una comprobación manual. El servidor web estático y el bot aleatorio local no inician esta API.

Diagnóstico

Síntoma	Acción
CUDA no disponible	Seleccionar GPU y reconectar antes de reejecutar entrenamiento.
Firma incompatible	Restaurar arquitectura y configuración del experimento o elegir un nombre de ejecución nuevo.
Memoria insuficiente	Revisar GPU, población y tamaño. Los lotes menores alivian activaciones, pero no todos los tensores del banco.
Modelo ausente o snapshot dañado	Ejecutar entrenamiento para guardar/publicar; revisar el mismo Drive y la integridad del bundle.
URL inaccesible o caducada	Reejecutar deployment, esperar pruebas públicas y copiar la nueva URL.
504 o motor ocupado	Esperar que finalice el trabajo anterior, revisar /stats y reintentar; se puede usar una dificultad menor.

Qué está comprobado y dónde leerlo

La descripción corresponde al código del repositorio al **26 de octubre de 2026**. Las cifras de tamaño provienen de la configuración y de `gpu_layout`; los ejemplos HTTP ilustran el contrato. No representan resultados de entrenamiento del usuario.

Alcance de la verificación

- **Red actual de 14.8 M:** se comprobaron configuración, conteo de parámetros y sintaxis de los notebooks. No se ejecutó una prueba de inferencia de esta arquitectura en la última ampliación; el entorno local carecía de PyTorch.
- **Pruebas locales anteriores:** la documentación del proyecto registra pruebas de evolución y recuperación con redes pequeñas en CPU, exportación y legalidad; pruebas API con pesos de prueba; y ejecución del deployment con HTTP local y servicios de Colab/túnel simulados.
- **Pendiente en el entorno real:** entrenamiento prolongado de esta red, consumo máximo de VRAM, velocidad CUDA/FP16, sincronización de Drive, túnel público efectivo y calidad ajedrecística del campeón.

Mapa de implementación

Archivo / celda	Responsabilidad
vendor/train.ipynb · train-config	Arquitectura, población, tiempos, nombre de ejecución y límites de recursos.
train-engine · GPU_SOURCE	Características, forward, estrategias de evolución, partidas, recuperación e inferencia.
train-engine · CORE_SOURCE	Escrituras atómicas y publicación del bundle de campeón.
train-run / train-export	Ejecución/reanudación y verificación del export.
vendor/deploy.ipynb	phase3-load: carga; phase3-app: contrato; phase3-server: servicio; phase3-tunnel: URL.
tests/test_notebooks.py	Persistencia, recuperación, publicación y separación de notebooks.
tests/test_api.py	Validación, legalidad, concurrencia, evaluación, errores y plazos HTTP.
tests/test_deployment_flow.py	Run all local, reejecución y cierre con proveedores simulados.

Compatibilidad y límites del documento

El modo `classic` sigue disponible: evoluciona 787 genes en CPU y usa búsqueda alfa-beta. No comparte la arquitectura neuronal ni su firma de checkpoint. Las definiciones opcionales de una segunda etapa siguen en el código, pero no forman parte del Run all predeterminado descrito aquí.

Los archivos con nombres históricos `cristopher_engine.py`, `cristopher_genome.json` y la carpeta de Drive se mantienen por compatibilidad. El nombre del proyecto es **Chess**.

> **Fuentes locales:** `vendor/train.ipynb`, `vendor/deploy.ipynb`, `README.md`, `PHASE2_README.md` y los tres archivos de pruebas anteriores. El HTML junto al PDF es la fuente editable del documento.